

Estrutura de Dados e Programação

Profa. Viviane Müller
(vmuller@unisc.br)

Estruturas de Dados

São formas de **representar** e relacionar os dados, de modo a tornar mais eficientes os algoritmos que **manipulam** esses dados.

Representar os dados - estudar várias formas de estruturar e armazenar os **dados** na memória principal.

Manipular os dados - estudar diversas **operações** que podemos aplicar sobre os dados.

Estruturas de Dados



Exemplo 1:

Problema: manipular um conjunto de fichas em um fichário.

Solução: organizar as fichas em ordem alfabética.

Operações Possíveis: inserir, retirar ou procurar uma ficha, etc.

Estrutura de Dados Correspondente: ?

Estruturas de Dados

Exemplo 2:



Problema: organizar as pessoas que querem, ser atendidas em um guichê.

Solução: colocar as pessoas em fila.

Operações Possíveis: a medida que uma pessoa é atendida no guichê, outra entra no final da fila. Não é permitido “furar” a fila, ou seja, entrar entre outras que já estão presentes.

Estrutura de Dados Correspondente: ?

Estruturas de Dados

Exemplo 3:



Problema: organizar um conjunto de pratos que estão sendo lavados, um a um, em um restaurante.

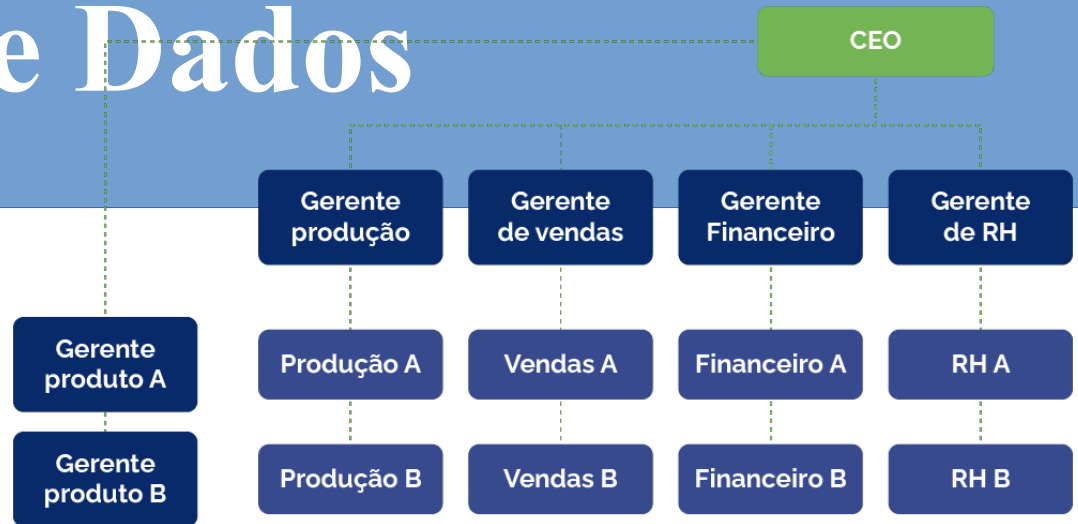
Solução: colocar os pratos empilhados.

Operações Possíveis: colocar um prato limpo no alto da pilha, retirar um prato do alto da pilha, etc.

Estrutura de Dados Correspondente: ?

Estruturas de Dados

Exemplo 4:



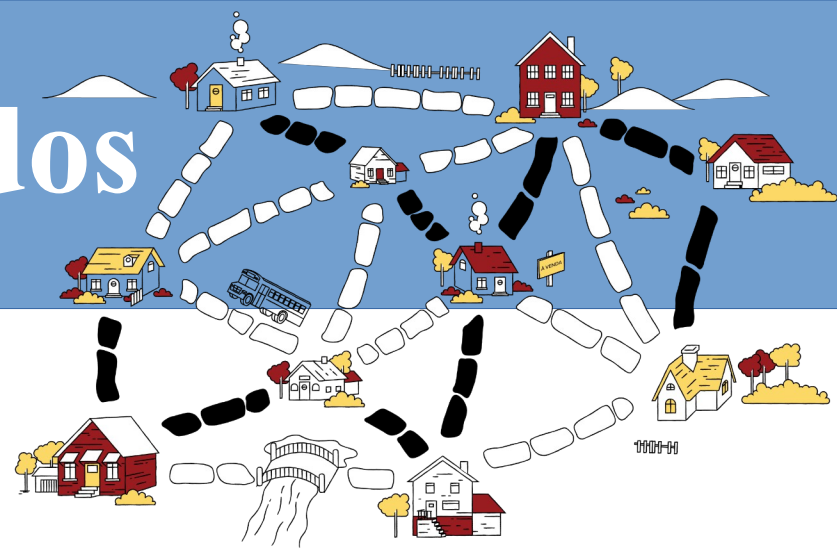
Problema: conseguir um modo de visualizar, o conjunto de pessoas que trabalham em uma empresa, tendo em conta sua função.

Solução: construir um organograma da empresa.

Operações Possíveis: inserir ou retirar certas funções, localizar uma pessoa, etc.

Estrutura de Dados Correspondente: ?

Estruturas de Dados



Exemplo 5:

Problema: estabelecer um trajeto para percorrer todas as capitais de um país.

Solução: utilizar um mapa que indique as rodovias existentes e estabelecer uma ordem possível para percorrer todas as cidades.

Operações Possíveis: encontrar um modo de percorrer todas as cidades, determinar o caminho mais curto para ir de uma cidade a outra, etc.

Estrutura de Dados Correspondente: ?

Estruturas de Dados

1 **LISTA** - seqüência de elementos dispostos em ordem.

2 **FILA** - seqüência de elementos dispostos em ordem com uma regra para a entrada e saída dos elementos (o 1º elemento que chega também é o 1º que sai da estrutura).

3 **PILHA** - seqüência de elementos dispostos em ordem, mas com uma regra para a entrada e saída dos elementos (o último que chega é o 1º que sai da estrutura).

Estruturas de Dados

4

ÁRVORE - estrutura de dados que caracteriza uma relação de hierarquia entre os elementos (uma pessoa não pode pertencer a dois departamentos diferentes, cada diretoria tem os seus próprios departamentos, etc.).

5

GRAFO - estrutura bastante genérica que organiza vários elementos, estabelecendo relações entre eles, dois a dois.

Tipos Abstratos de Dados (TAD)

Um Tipo Abstrato de Dados (TAD) é um modelo que define quais operações podem ser realizadas sobre um conjunto de dados, sem se preocupar com a forma como esses dados são armazenados ou implementados.

Em outras palavras, um TAD descreve o que uma estrutura faz, e não como ela faz.

Tipos Abstratos de Dados (TAD)

Exemplo do dia a dia:

Caixa eletrônico

Você pode realizar operações como:

- sacar dinheiro;
- consultar saldo;
- fazer um depósito;
- transferir valores.

Tipos Abstratos de Dados (TAD)

Você sabe quais operações o caixa oferece, mas não precisa conhecer como o banco processa essas operações internamente.

O caixa eletrônico funciona como um Tipo Abstrato de Dados: ele disponibiliza uma interface de operações e esconde os detalhes da implementação.

Tipos Abstratos de Dados (TAD)

Em C, normalmente representamos um TAD utilizando uma estrutura (`struct`) e um conjunto de funções responsáveis por manipular os dados.

```
#include <stdio.h>

typedef struct {
    int dados[MAX];
    int topo;
} Pilha;

void inicializar(Pilha *p) {
    p->topo = -1;
}
```

Tipos Abstratos de Dados (TAD)

Vantagens dos TADs

Organização: separa os dados das operações.

Reutilização: o mesmo TAD pode ser utilizado em diferentes programas.

Facilidade de manutenção: mudanças na implementação não afetam quem utiliza o TAD.

Modularização: divide o programa em componentes menores e mais fáceis de entender.

Abstração: permite que o programador trabalhe com conceitos, sem se preocupar inicialmente com detalhes de implementação.

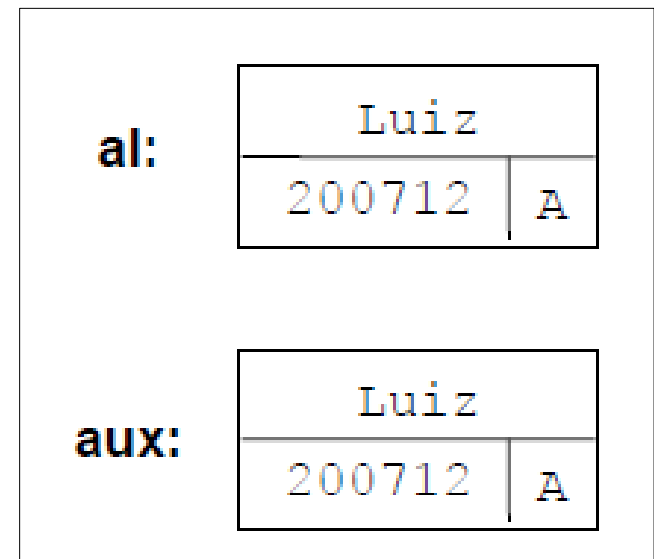
Registros



```
struct conta{  
    int nro_conta;  
    char nome[50];  
    float saldo;  
};
```

Estruturas em C

```
struct Aluno {  
    char nome[100];  
    int matricula;  
    char conceito;  
};  
  
main() {  
    struct Aluno al, aux;  
    al.nome = "Luiz"  
    al.matricula = 200712;  
    al.conceito = 'A';  
    aux = al;  
    printf("%s", aux.nome);  
}
```



Representação física dos dados

A representação física dos dados define como os elementos de uma estrutura de dados são armazenados na memória do computador.

As duas formas mais utilizadas são:

- Representação sequencial
- Representação encadeada.

Representação física dos dados

Representação Sequencial

Na representação sequencial, os elementos são armazenados em posições consecutivas da memória, normalmente utilizando um vetor.

Representação física dos dados

Vantagens

- Acesso rápido aos elementos.
- Implementação simples.
- Menor uso de memória (não precisa armazenar ponteiros).

Desvantagens

- Inserções e remoções podem exigir o deslocamento de vários elementos.
- Possui tamanho limitado quando implementada com vetores estáticos.

Representação física dos dados

Representação Encadeada

Na representação encadeada, os elementos não precisam estar armazenados em posições consecutivas da memória. Cada elemento contém o dado e um ponteiro para o próximo elemento da estrutura. Cada bloco é chamado de nó.

Representação física dos dados

Vantagens

- Inserções e remoções são mais eficientes.
- Não possui tamanho fixo.
- Utiliza a memória conforme a necessidade.

Desvantagens

- Acesso aos elementos é sequencial (não existe acesso direto por índice).
- Consome mais memória devido aos ponteiros.

Representação física dos dados

Representação Sequencial	Representação Encadeada
Utiliza vetor	Utiliza nós e ponteiros
Elementos ficam em posições consecutivas	Elementos podem estar em qualquer posição da memória
Acesso direto por índice	Acesso sequencial
Inserções e remoções podem ser lentas	Inserções e remoções são mais rápidas
Tamanho geralmente fixo	Tamanho dinâmico

Alocação de Memória

Ao criar um programa em C usualmente temos que especificar, antes de começar a executar o programa, as variáveis que vamos usar, reservando assim um espaço na memória.

As variáveis que são alocadas em posições fixas da memória são chamadas de *variáveis estáticas*, e as variáveis que não possuem uma posição fixa, e que são criadas e destruídas durante a execução do programa, são chamadas de *variáveis dinâmicas*.

Alocação de Memória

Alocação Estática: os dados tem um tamanho fixo e estão organizados seqüencialmente na memória do computador.

O espaço de memória para as variáveis é reservado no início da execução

Um exemplo típico de alocação estática são as variáveis globais, e os vetores (estáticos).

Alocação de Memória

Alocação Dinâmica: os dados não precisam ter um tamanho fixo, pois podemos definir para cada dado quanto de memória desejamos usar.

O espaço de memória para as variáveis pode ser alocado dinamicamente durante a execução do programa.

Os espaços de memória (blocos) não precisam estar necessariamente organizados de maneira seqüencial, podendo estar distribuídos de forma esparsa na memória do computador.

Alocação de Memória

A memória alocada dinamicamente é acessada através de **apontadores** (*pointers*) que na verdade são variáveis que armazenam o endereço de uma área de memória.

Basicamente, o programa aloca e desaloca porções de memória durante a execução.